

Serverless Architectures TASK – 3

1. List 3 real-world apps you use (like Zomato, Instagram, or Paytm) and describe one scenario in each where using serverless architecture would improve cost efficiency or scalability.

Ans :

1. Zomato

- **Scenario:** Handling sudden spikes in orders during festivals or major events.
- **Serverless benefit:** Functions can automatically scale up when order traffic increases and scale down afterward. Zomato would pay mainly for the compute time actually used instead of keeping large servers running continuously.

2. Instagram

- **Scenario:** Generating image/video thumbnails and processing uploaded media.
- **Serverless benefit:** Each upload can trigger a serverless function to resize, compress, or convert the media. Thousands of uploads can be processed concurrently without manually provisioning additional servers.

3. Paytm

- **Scenario:** Sending transaction notifications such as SMS, email, or push notifications.
- **Serverless benefit:** A payment event can trigger a serverless function that processes and sends the notification. The system can automatically handle high transaction volumes while avoiding always-on infrastructure for relatively short-lived tasks.

2.Create a simple REST API for a 'Favorite Songs' list using AWS Lambda and API Gateway — implement a Lambda function in your preferred language that returns a hardcoded list of 3 song names when accessed via an HTTP GET request.

Ans :

1. Log in to your **AWS Management Console**.
2. Open **AWS Lambda**.
3. Click **Create function**.
4. Select **Author from scratch**.
5. Enter the function name:
6. favorite-songs-api
7. Select **Python 3.x** as the runtime.
8. Leave the other settings as default and click **Create function**.
9. In the Lambda code editor, replace the existing code with:
10. import json
- 11.
12. def lambda_handler(event, context):
13. songs = [
14. "Blinding Lights - The Weeknd",
15. "Shape of You - Ed Sheeran",
16. "Levitating - Dua Lipa"
17.]
- 18.
19. return {
20. "statusCode": 200,

21. "headers": {
22. "Content-Type": "application/json"
23. },
24. "body": json.dumps({
25. "favorite_songs": songs
26. })
27. }
28. Click **Deploy** to save the Lambda function.
29. Open **Amazon API Gateway** from the AWS Console.
30. Click **Create API**.
31. Under **HTTP API**, click **Build**.
32. Add your Lambda function favorite-songs-api as the integration.
33. Configure the route:

Method: GET

Path: /songs

15. Continue through the setup and click **Create**.
16. API Gateway will provide an **Invoke URL**, similar to:
`https://abc123.execute-api.us-east-1.amazonaws.com`
17. Your complete API endpoint will be:
`https://abc123.execute-api.us-east-1.amazonaws.com/songs`
18. Open the endpoint in a browser, or test it using:
`curl https://abc123.execute-api.us-east-1.amazonaws.com/songs`
19. You should receive:

```
{  
  "favorite_songs": [  
    "Blinding Lights - The Weeknd",  
    "Shape of You - Ed Sheeran",  
    "Levitating - Dua Lipa"  
  ]  
}
```

20. The final architecture is:

User/Client



API Gateway



GET /songs



AWS Lambda



Hardcoded list of 3 songs

3. Modify your Lambda function to accept a POST request that adds a new song name (sent in the request body) to an in-memory array, and returns the updated list.
Hint: Use a global variable in your Lambda function to simulate the array during the same execution (stateless between invocations).

Ans :

```
// Global variable — persists only while the Lambda execution
environment is reused
```

```
let songs = [];
```

```
exports.handler = async (event) => {
```

```
    const method = event.requestContext?.http?.method ||
event.httpMethod;
```

```
    if (method === "POST") {
```

```
        let body;
```

```
        try {
```

```
            body = typeof event.body === "string"
```

```
                ? JSON.parse(event.body)
```

```
                : event.body;
```

```
        } catch (error) {
```

```
            return {
```

```
                statusCode: 400,
```

```
                body: JSON.stringify({ error: "Invalid JSON body" })
```

```
            };
```

```
        }
```

```
        const songName = body?.songName;
```

```
if (!songName) {  
    return {  
        statusCode: 400,  
        body: JSON.stringify({ error: "songName is required" })  
    };  
}
```

```
songs.push(songName);
```

```
return {  
    statusCode: 200,  
    body: JSON.stringify(songs)  
};  
}
```

```
return {  
    statusCode: 405,  
    body: JSON.stringify({ error: "Method not allowed" })  
};  
};
```

4. Configure an API Gateway endpoint to trigger your Lambda function for both GET and POST requests, and test both endpoints using Postman or curl.
Hint: Make sure to set the correct HTTP method mappings in API Gateway.

Ans :

1. Configure API Gateway

In the AWS Console:

1. Open **API Gateway**.
2. Create an **HTTP API**.
3. Add your existing Lambda function as the integration.
4. Create a route for:
 - GET /songs
 - POST /songs
5. Make sure **both routes point to the same Lambda function**.
6. Deploy the API and copy the generated **Invoke URL**, for example:
7. `https://abc123.execute-api.ap-south-1.amazonaws.com`

Your endpoints will then be:

GET `https://abc123.execute-api.ap-south-1.amazonaws.com/songs`

POST `https://abc123.execute-api.ap-south-1.amazonaws.com/songs`

2. Make sure Lambda handles GET

Add a GET branch to the Lambda from the previous step:

```
let songs = [];
```

```
exports.handler = async (event) => {  
    const method = event.requestContext?.http?.method ||  
    event.httpMethod;
```

```
if (method === "GET") {  
  return {  
    statusCode: 200,  
    body: JSON.stringify(songs)  
  };  
}
```

```
if (method === "POST") {  
  let body;  
  
  try {  
    body = typeof event.body === "string"  
      ? JSON.parse(event.body)  
      : event.body;  
  } catch (error) {  
    return {  
      statusCode: 400,  
      body: JSON.stringify({ error: "Invalid JSON body" })  
    };  
  }  
}
```

```
const songName = body?.songName;
```

```
if (!songName) {
```

```
    return {  
        statusCode: 400,  
        body: JSON.stringify({ error: "songName is required" })  
    };  
}
```

```
songs.push(songName);
```

```
    return {  
        statusCode: 200,  
        body: JSON.stringify(songs)  
    };  
}
```

```
    return {  
        statusCode: 405,  
        body: JSON.stringify({ error: "Method not allowed" })  
    };  
};
```

3. Test GET with curl

Initially, the array should be empty:

```
curl -X GET "https://abc123.execute-api.ap-south-  
1.amazonaws.com/songs"
```

Expected response:

[]

4. Test POST with curl

Add a song:

```
curl -X POST \  
  "https://abc123.execute-api.ap-south-1.amazonaws.com/songs" \  
  -H "Content-Type: application/json" \  
  -d '{"songName":"Bohemian Rhapsody"}'
```

Expected response:

```
["Bohemian Rhapsody"]
```

Add another:

```
curl -X POST \  
  "https://abc123.execute-api.ap-south-1.amazonaws.com/songs" \  
  -H "Content-Type: application/json" \  
  -d '{"songName":"Imagine"}'
```

Expected response:

```
["Bohemian Rhapsody","Imagine"]
```

Then test GET again:

```
curl -X GET "https://abc123.execute-api.ap-south-1.amazonaws.com/songs"
```

Expected:

```
["Bohemian Rhapsody","Imagine"]
```

5. Test with Postman

Create two requests using the same base URL.

GET

GET https://abc123.execute-api.ap-south-1.amazonaws.com/songs

No body is required.

POST

POST https://abc123.execute-api.ap-south-1.amazonaws.com/songs

Under **Body** → **raw** → **JSON**, enter:

```
{  
  "songName": "Hotel California"  
}
```

5. Pick a real-time data processing use case from an app you use (for example, live IPL score updates in Cricbuzz or real-time chat in WhatsApp). Explain in 3-4 lines how you would design a serverless workflow for this using Lambda, API Gateway, and any other AWS service if needed.

Ans :

- **API Gateway** receives score-update requests from the cricket scoring system and triggers **AWS Lambda**.
- Lambda validates and processes each update, then stores the latest match state in **DynamoDB**.
- **DynamoDB Streams** can trigger another Lambda whenever the score changes, which publishes the update through **API Gateway/WebSocket** to connected users.
- This provides a serverless, scalable workflow where users receive near real-time score updates without managing servers.